

Final Project for Computational Thinking Course

Pairwise Global Alignment Using Affine Gap Penalties

Xianxin Long

2230416029

December 2025

Table of Contents

The String Alignment Problem

The Needleman-Wunsch Algorithm

Program Description

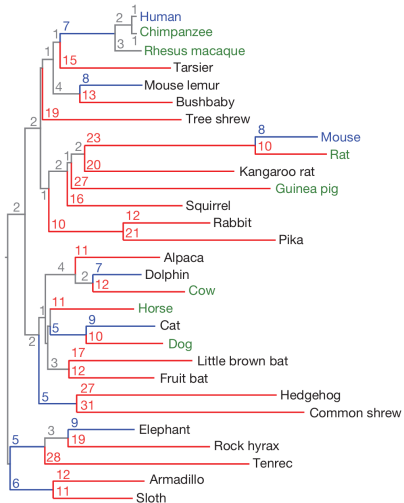
Table of Contents

The String Alignment Problem

The Needleman-Wunsch Algorithm

Program Description

Why Compare Biological Sequences



The Definition of the Pairwise Global Alignment Problem

Assume $\mathcal{A}$ is an alphabet. Given 2 strings $V = v_1 v_2 \dots v_m$ and $W = w_1 w_2 \dots w_n$, where $v_i, w_j \in \mathcal{A}$, $i = 1, \dots, m$, $j = 1, \dots, n$. Construct an alignment π that introduces gaps ("-") to the 2 strings and preserves the order to make the 2 aligned strings $V' = v'_1 v'_2 \dots v'_L$ and $W' = w'_1 w'_2 \dots w'_L$ the same length L . We want to find the optimal alignment that maximizes the scoring function $S(\pi) = \sum_{k=1}^L \xi(a'_k, b'_k)$, where $\xi : (\mathcal{A} \cup \{-\}) \times (\mathcal{A} \cup \{-\}) \rightarrow \mathbb{R}$.

ATGCTTAG ATGC-TTAG
TGCATTAA -TGCATTAA

Scoring Functions

Linear penalties:

$$\xi(x, y) = \begin{cases} \omega & \text{if } x \in \mathcal{A}, y \in \mathcal{A}, x = y \\ -\mu & \text{if } x \in \mathcal{A}, y \in \mathcal{A}, x \neq y \\ -d & \text{if } x = - \text{ or } y = - \end{cases}$$

Which one is better?

Affine gap penalties:

$$\xi(x, y) = \begin{cases} \omega & \text{if } x \in \mathcal{A}, y \in \mathcal{A}, x = y \\ -\mu & \text{if } x \in \mathcal{A}, y \in \mathcal{A}, x \neq y \\ -\sigma - \epsilon & \text{if } x = - \text{ or } y = -, \text{ and is the first position of the gaps} \\ -\epsilon & \text{if } x = - \text{ or } y = -, \text{ and is subsequent in the gaps} \end{cases}$$

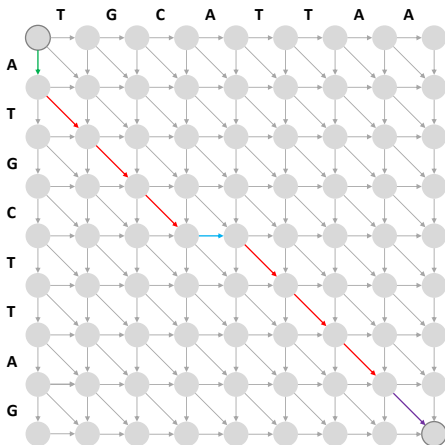
GATCCAG

GATCCAG

GA-C-AG

GA--CAG

From String to Graph



We could use an edge-weighted directed acyclic graph to represent the alignment:

- ▶ $\downarrow$: insertion in string W
- ▶ $\searrow$: match/mismatch
- ▶ $\rightarrow$: insertion in string V

For a grid with $m+1$ rows and $n+1$ columns, there are $\sum_{k=0}^{\min(m+1, n+1)} \binom{m+1}{k} \binom{n+1}{k} 2^k$ paths (Delannoy number).

Dynamic Programming

When to use Dynamic Programming (DP) ?

- ▶ 1. Optimal substructure: The optimal solution to a problem consists of the optimal solutions to its sub-problems.
- ▶ 2. Overlapping subproblems: During the recursive solution process, the sub-problems will be recalculated multiple times.
- ▶ 3. Markov property: Once the current state is determined, subsequent decisions will only rely on the current state.

Almost every DP problem could be abstracted into the problem of finding the longest or shortest path in an edge-weighted DAG!

Table of Contents

The String Alignment Problem

The Needleman-Wunsch Algorithm

Program Description

Dynamic Programming for Linear Penalty Alignment

Let $S \in \mathbb{R}^{(m+1) \times (n+1)}$,

- ▶ 1. Initialization:

$$S_{i,1} = (i-1) \cdot d, \quad S_{1,j} = (j-1) \cdot d$$

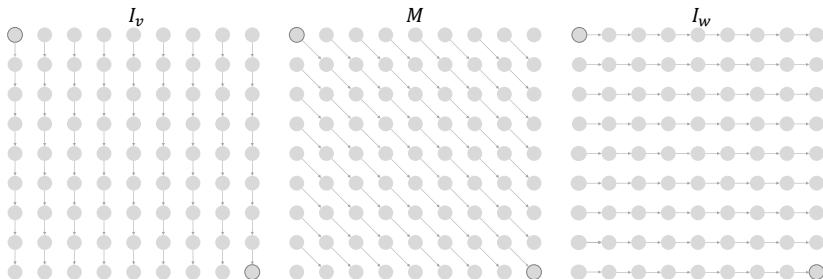
- ▶ 2. Solve sub-problems:

$$S_{i+1,j+1} = \max \begin{cases} S_{i,j+1} - d \\ S_{i+1,j} - d \\ S_{i,j} + \omega & \text{if } v_{i+1} = w_{j+1} \\ S_{i,j} - \mu & \text{if } v_{i+1} \neq w_{j+1} \end{cases}$$

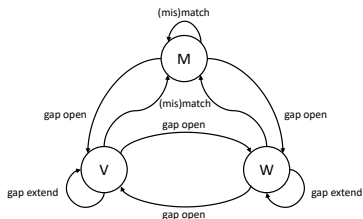
where $i \in \{1, 2, \dots, m\}$, $j \in \{1, 2, \dots, n\}$.

- ▶ 3. Trace-back: from the sink node ($S_{m+1,n+1}$) to source node ($S_{1,1}$), find the optimal path (alignment).

Manhattan in 3 Layers



- insertion in string V
- match/mismatch
- insertion in string W



Initialization

Let $l_{v_{1,1}} = M_{1,1} = l_{w_{1,1}} = 0$,

Col initialization:

- ▶ $l_{v_{i,1}} = -\sigma - (i - 2) \cdot \epsilon$
- ▶ $M_{i,1} = l_{w_{i,1}} = -\infty$

Row initialization:

- ▶ $l_{w_{1,j}} = -\sigma - (j - 2) \cdot \epsilon$
- ▶ $M_{1,j} = l_{v_{1,j}} = -\infty$

where $i \in \{2, \dots, m + 1\}$, $j \in \{2, \dots, n + 1\}$.

Recurrence Relations

$$I_{v_{i+1},j+1} = \max \begin{cases} I_{v_i,j+1} - \epsilon \\ M_{i,j+1} - \sigma - \epsilon \end{cases}$$

$$I_{w_{i+1},j+1} = \max \begin{cases} I_{w_{i+1},j} - \epsilon \\ M_{i+1,j} - \sigma - \epsilon \end{cases}$$

$$M_{i+1,j+1} = \max \begin{cases} I_{v_{i+1},j+1} \\ M_{i,j+1} + \omega & \text{if } v_{i+1} = w_{j+1} \\ M_{i,j+1} - \mu & \text{if } v_{i+1} \neq w_{j+1} \\ I_{w_{i+1},j+1} \end{cases}$$

where $i \in \{1, 2, \dots, m\}$, $j \in \{1, 2, \dots, n\}$.

Trace-back

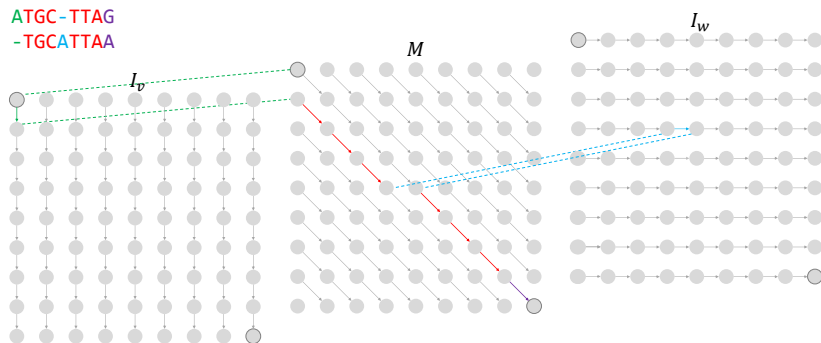


Table of Contents

The String Alignment Problem

The Needleman-Wunsch Algorithm

Program Description

Usage

Example

```
needle -a ATGC -b ATCG -m 2 -M 1 -o 1 -e 0.5
```

- ▶ -a sequence A
- ▶ -b sequence B
- ▶ -m match score
- ▶ -M mismatch score
- ▶ -o gap open score
- ▶ -e gap extend score

Important restricts

- ▶ $\mathcal{A} = \{a, b, \dots, z\} \cup \{A, B, \dots, Z\}$, case insensitive.
- ▶ The maximum length of the input sequence is 10000.

Output

```
(base) PS C:\Users\lxx\Desktop> needle \  
-a ATGAGTCTCTCTGATAAGGACAAGGCTGCTGTGAAAGCCCTATGG \  
-b CTGTCTCCTGCCGACAAGACCAACGTCAAGGCCGCCTGGGGTAAG \  
-m 5 -M 4 -o 10 -e 0.5  
#####  
# Program: needle  
# Date: Sun Dec 21 21:08:44 2025  
# Command line: needle -a ATGAGTCTCTCTGATAAGGACAAGGCTGCTGTGAAAGCCCTATGG -b  
CTGTCTCCTGCCGACAAGACCAACGTCAAGGCCGCCTGGGGTAAG -m 5.0 -M 4.0 -o 10.0 -e 0.5  
#####  
#=====
```

# Length:	57
# Identity:	28
# Gaps:	24
# Score:	38.0
#=====	

Alignments:

SeqA	1	ATGAGTCTCTCTG	---	ATAAG	-----	GACAAGGCTGC	-	TGTG	--	AAAGCCCTATGG	45	
SeqB	1	CTG	--	TCTC	-	CTGCCGACAAG	ACCAACGTCA	AGGCCGCCT	GGGGTAA	-----	G	45

Thanks!